CAHIER DES CHARGES TECHNIQUE

Projet GRESI

Architecture, conventions et plan de démarrage

Élément	Valeur
Projet	GRESI — inscriptions aux activités et distribution des repas, Semaine de l'Industrie
Destinataires	Équipe de développement (3 développeurs)
Nature	Référence technique : architecture, nommage, modèle de données, démarrage
Document associé	Cahier des charges fonctionnel V2.0
Version	2.0 (remplace la v1.0)
Date	6 octobre 2026

Semaine de l'Industrie — École Supérieure d'Industrie (ESI), INPHB

Historique et objet du document

Version	Date	Évolutions
1.0	Sept. 2026	Cahier des charges techniques initial : stack, rôles, écrans, modèle de données, cycle du QR, conventions.
2.0	**6 oct. 2026**	Refonte complète : architecture monorepo, nommage détaillé, modèle de données v2, variables d'environnement, plan de démarrage (semaine 0), règles d'équipe.

Ce document est la **référence technique** de GRESI. Il complète le *Cahier des charges fonctionnel V2.0*, qui décrit **quoi** construire ; celui-ci décrit **comment** l'organiser : où ranger chaque fichier, comment nommer chaque élément, comment est structurée la base, et tout ce qui doit être prêt **avant la première ligne de code fonctionnel**.

> **Règle d'or**
> En cas de doute sur un nom, un emplacement ou une convention, **ce document fait foi**. S'il ne répond pas à la question, on décide en équipe, on consigne la décision dans `docs/decisions/` et on met ce document à jour. Aucune convention n'est décidée « dans son coin ».

Sommaire

Historique et objet du document 2
1. Principes d'architecture 4
2. Vue d'ensemble 5
  2.1 Environnements 5
  2.2 Décisions d'architecture (ADR) 5
3. Arborescence du dépôt 6
  3.1 Rôle de chaque dossier 7
  3.2 Anatomie d'un dossier « feature » 8
  3.3 Routes de l'application 8
4. Conventions de nommage 10
  4.1 Langue 10
  4.2 Git et GitHub 10
  4.3 Base de données 10
  4.4 TypeScript et React 11
  4.5 Styles 12
5. Variables d'environnement et secrets 13
6. Modèle de données v2 14
  6.1 Types énumérés 14
  6.2 Tables 15
  6.3 Fonctions RPC et Edge Functions 16
  6.4 Matrice des droits (RLS) 17
  6.5 Ce qui change par rapport aux scripts existants 18
  6.6 Découpage en migrations 18
7. Flux clés 19
  7.1 Scan continu à plusieurs distributeurs 19
  7.2 Création de compte étudiant 20
8. Plan de démarrage — semaine 0 (6 → 11 oct.) 21
  8.1 Réunion de lancement (mer. 7, 2 h) 21
  8.2 Comptes et espaces de travail (jeu. 8) 21
  8.3 Dépôt GitHub et squelette (ven. 9) 22
  8.4 Base de données (sam. 10) 23
  8.5 Déploiement et CI (dim. 11) 23
  8.6 Revue finale (dim. 11, 30 min, les 3) 23
  8.7 Definition of Ready — à cocher avant de coder 23
9. Règles d'équipe 25
  9.1 Flux Git 25
  9.2 Règles spécifiques à la base 25
  9.3 Definition of Done 25
  9.4 Rituels 25
  9.5 Communication et décisions 25
10. Répartition des responsabilités 26
Annexe A. Commandes utiles 27
Annexe B. Fichiers modèles 28
Annexe C. Points ouverts 30

Sous Word : clic droit sur le sommaire → « Mettre à jour les champs » si les numéros de page n'apparaissent pas.

1. Principes d'architecture

Principe	Conséquence concrète
Une seule application web	Participants, admins et distributeurs utilisent la même application React. Le scan est une page `/scan` installable en PWA. Un seul déploiement, une seule base de code à maintenir.
La base est la source de vérité	Toutes les règles de gestion (RG1–RG16) sont garanties en base (contraintes, triggers, RPC). L'interface les rappelle mais ne les garantit jamais seule.
Écritures sensibles = RPC	Choisir une activité, valider un scan, régénérer un QR : uniquement via des fonctions `SECURITY DEFINER` qui vérifient le rôle. Pas d'`insert`/`update` direct depuis le client sur ces tables.
RLS partout	Toute table du schéma `public` a la Row Level Security activée, avec des politiques écrites dans la même migration que la table.
Types générés, jamais écrits	Les types TypeScript de la base sont générés par la CLI Supabase dans `packages/shared`. Personne ne les modifie à la main.
Migrations versionnées	Le schéma n'évolue que par des fichiers de migration dans `supabase/migrations`. Aucune modification manuelle dans le tableau de bord Supabase, sauf en local pour expérimenter.
Rien en dur	Dates, horaires, fenêtre de scan : table `parametres_evenement`. URL et clés : variables d'environnement.
Découpage par domaine	Le code web est rangé par fonctionnalité métier (`features/scan`, `features/activites`…), pas par type technique. Un développeur travaille principalement dans « son » dossier.

2. Vue d'ensemble

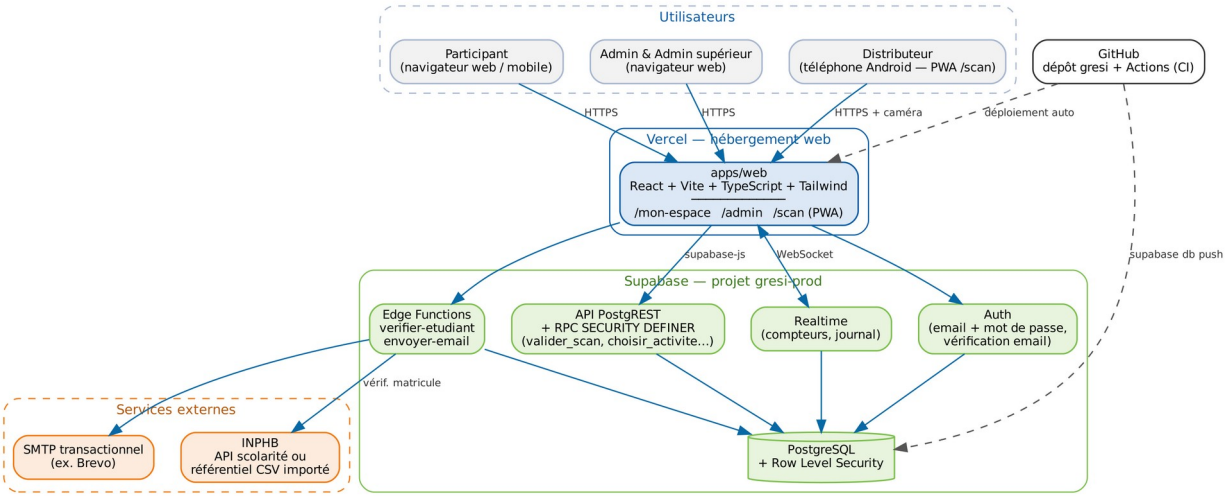


Figure 1 — Architecture générale

2.1 Environnements

Environnement	Base Supabase	Web (Vercel)	Branche Git	Usage
Local	Supabase local (Docker)	`pnpm dev` (localhost:5173)	branche de travail	Développement quotidien, données de `seed.sql`
Préproduction	projet `gresi-dev`	déploiement *Preview*	`main` + PR	Intégration, démos, tests sur téléphone
Production	projet `gresi-prod`	déploiement *Production*	`production`	Évènement réel ; données réelles uniquement ici

2.2 Décisions d'architecture (ADR)

Chaque décision structurante est consignée dans `docs/decisions/ADR-NNN-titre.md` (contexte, décision, alternatives, conséquences). Les premières sont :

N°	Décision	Statut
ADR-001	Monorepo pnpm (`apps/`, `packages/`, `supabase/`)	Acceptée
ADR-002	Supabase comme backend (Auth, PostgreSQL, RLS, RPC, Realtime, Edge Functions)	Acceptée
ADR-003	Scan en PWA dans l'application web ; Expo (`apps/scan-mobile`) en plan B	À confirmer fin S1
ADR-004	QR = jeton opaque UUID préfixé `GRESI:1:`, régénérable	Acceptée
ADR-005	Vérification des étudiants via `etudiants_reference` (alimentée par import CSV ou par l'API INPHB)	Acceptée
ADR-006	Vocabulaire métier en français, motifs techniques en anglais (§4.1)	Acceptée
ADR-007	Branches `main` (préprod) et `production` ; migrations prod manuelles	Acceptée
ADR-008	Emails limités à l'Auth + annulations ; annonces dans l'application	Acceptée

3. Arborescence du dépôt

Un seul dépôt GitHub, `gresi`, organisé en monorepo avec **pnpm workspaces**. Les noms de dossiers ci-dessous sont **définitifs** : ne pas en créer d'autres au premier niveau sans ADR.

```
gresi/
├── .github/
│   ├── workflows/
│   │   ├── ci.yml                    # lint, types, build, tests SQL (chaque PR)
│   │   └── deploy-db-prod.yml        # migrations vers gresi-prod (manuel)
│   ├── ISSUE_TEMPLATE/
│   │   ├── fonctionnalite.md
│   │   └── bug.md
│   ├── PULL_REQUEST_TEMPLATE.md
│   └── CODEOWNERS
├── apps/
│   └── web/                          # UNE seule application : participant, admin, scan
│       ├── public/
│       │   ├── icons/                # icone-192.png, icone-512.png (PWA)
│       │   └── sons/                 # valide.mp3, refuse.mp3
│       ├── src/
│       │   ├── app/
│       │   │   ├── App.tsx
│       │   │   ├── router.tsx        # déclaration de TOUTES les routes
│       │   │   ├── providers.tsx     # QueryClient, session
│       │   │   └── guards/
│       │   │       └── RequireRole.tsx
│       │   ├── features/             # un dossier = un domaine métier
│       │   │   ├── auth/
│       │   │   ├── activites/
│       │   │   ├── espace-participant/
│       │   │   ├── scan/
│       │   │   ├── annonces/
│       │   │   └── admin/
│       │   │       ├── statistiques/
│       │   │       ├── participants/
│       │   │       ├── personnel/
│       │   │       ├── gestion-activites/
│       │   │       └── journal/
│       │   ├── components/
│       │   │   ├── ui/               # Bouton, Carte, ChampTexte, Modale, Badge…
│       │   │   └── layout/           # MiseEnPageParticipant, MiseEnPageAdmin…
│       │   ├── lib/
│       │   │   ├── supabase.ts       # client Supabase unique
│       │   │   ├── env.ts            # lecture + validation des VITE_*
│       │   │   ├── queryClient.ts
│       │   │   └── format.ts         # dates, masquage du matricule
│       │   ├── styles/
│       │   │   └── index.css         # Tailwind + couleurs GRESI (@theme)
│       │   ├── main.tsx
│       │   └── vite-env.d.ts
│       ├── .env.example
│       ├── index.html
│       ├── package.json
│       ├── tsconfig.json
│       ├── vercel.json               # réécriture SPA vers index.html
│       └── vite.config.ts            # React + Tailwind + PWA
├── packages/
│   └── shared/                       # code partagé (et réutilisable par Expo si plan B)
```

```
│       ├── src/
│       │   ├── types/
│       │   │   ├── database.types.ts  # GÉNÉRÉ par Supabase — ne jamais éditer
│       │   │   └── domaine.ts         # ResultatScan, ParticipantResume…
│       │   ├── schemas/               # validation zod des formulaires
│       │   │   ├── inscription.ts
│       │   │   └── activite.ts
│       │   ├── constantes.ts          # CRENEAUX, ROLES, PREFIXE_QR…
│       │   └── index.ts
│       ├── package.json
│       └── tsconfig.json
├── supabase/
│   ├── config.toml
│   ├── migrations/                    # AAAAMMJJHHMMSS_nom.sql (voir §6.6)
│   ├── functions/
│   │   ├── _shared/                   # cors.ts, clientAdmin.ts, reponses.ts
│   │   ├── verifier-etudiant/index.ts
│   │   ├── creer-personnel/index.ts
│   │   ├── creer-participant-sur-place/index.ts
│   │   └── envoyer-email/index.ts
│   ├── templates/                     # emails Auth en français
│   │   ├── confirmation.html
│   │   └── reinitialisation.html
│   ├── tests/                         # pgTAP, un fichier par règle
│   │   ├── rg01_un_choix_par_creneau.test.sql
│   │   ├── rg05_creneau_utilise.test.sql
│   │   └── rg12_scans_simultanes.test.sql
│   └── seed.sql                       # données de test (dev uniquement)
├── scripts/
│   ├── importer-etudiants.ts          # CSV scolarité → etudiants_reference
│   └── test-charge/
│       └── valider-scan.js            # script k6
├── docs/
│   ├── cahier-des-charges/            # .docx et .pdf versionnés
│   ├── decisions/                     # ADR-001-monorepo.md, ADR-002-…
│   ├── comptes-rendus/                # AAAA-MM-JJ-objet.md
│   ├── guides/                        # guide-distributeur.md, guide-admin.md…
│   └── maquettes/
├── .editorconfig
├── .gitignore
├── .nvmrc                             # version de Node
├── .prettierrc
├── eslint.config.js
├── package.json                       # scripts racine (dev, lint, db:*)
├── pnpm-workspace.yaml
├── tsconfig.base.json
├── README.md                          # démarrage en 10 minutes
└── CONTRIBUTING.md                    # règles d'équipe (§9)
```

3.1 Rôle de chaque dossier

Dossier	Contenu	Référent
`apps/web/src/app`	Point d'entrée, routeur, providers, gardes de rôle. Rarement modifié.	Dev web
`apps/web/src/features/*`	Le code métier, un dossier par domaine (structure §3.2).	Selon le domaine

Dossier	Contenu	Référent
`apps/web/src/components`	Composants **sans logique métier**, réutilisés par plusieurs domaines.	Dev web
`apps/web/src/lib`	Clients et utilitaires techniques (Supabase, React Query, formatage).	Dev web
`packages/shared`	Types générés, types métier, constantes, schémas zod partagés.	Stéphane
`supabase/migrations`	Schéma, contraintes, RLS, fonctions — seule source de vérité de la base.	Stéphane
`supabase/functions`	Edge Functions (opérations nécessitant la clé secrète ou un appel externe).	Stéphane
`supabase/tests`	Tests pgTAP, un fichier par règle de gestion.	Stéphane
`scripts`	Outils ponctuels : import du référentiel, test de charge.	Stéphane
`docs`	Cahiers des charges, ADR, comptes rendus, guides, maquettes.	Tous

3.2 Anatomie d'un dossier « feature »

Tous les domaines suivent la même structure. Exemple avec `features/scan` :

```
features/scan/
├── api.ts                    # appels Supabase de ce domaine UNIQUEMENT
├── components/
│   ├── CameraScanner.tsx
│   ├── EcranResultat.tsx     # plein écran vert / rouge
│   ├── CompteurSalle.tsx     # « validés / inscrits » en temps réel
│   └── RechercheManuelle.tsx
├── hooks/
│   ├── useScanner.ts         # cycle lecture → appel → affichage → reprise
│   ├── useAntiRebond.ts
│   └── useCompteurTempsReel.ts
├── pages/
│   └── ScanPage.tsx
└── types.ts                  # types locaux au domaine (si besoin)
```

- Un composant de `features/scan` **n'importe jamais** depuis `features/admin` : ce qui est partagé remonte dans `components/` ou `packages/shared`.
- Les appels à Supabase sont **uniquement** dans `api.ts` ; les composants passent par des hooks React Query.

3.3 Routes de l'application

Route	Page	Accès
`/connexion`, `/inscription`, `/mot-de-passe-oublie`	ConnexionPage, InscriptionPage, MotDePasseOubliePage	Public
`/mon-espace`	Tableau de bord participant (« Mes créneaux »)	Participant
`/mon-espace/activites`	Liste et choix des activités	Participant
`/mon-espace/qr`	QR code plein écran	Participant
`/scan`	Sélection de l'activité puis scan continu	Distributeur
`/admin`	Statistiques	Admin, Admin sup.

Route	Page	Accès
`/admin/participants`, `/admin/annonces`	Listes et annonces	Admin, Admin sup.
`/admin/activites`, `/admin/personnel`, `/admin/journal`	Gestion et audit	Admin sup.

4. Conventions de nommage

4.1 Langue

> **Règle de langue (ADR-006)**
>
> **Vocabulaire métier en français, sans accents** : tables, colonnes, fonctions SQL, composants et variables métier (`inscriptions`, `valider_scan`, `EcranResultat`, `activiteEnCours`).
>
> **Motifs techniques standards en anglais** : `use…` pour les hooks, suffixe `Page`, `on…`/`handle…` pour les événements, dossiers `components`, `hooks`, `lib`.
>
> Exception documentée : les fonctions d'aide existantes `is_admin()`, `is_admin_superieur()`, `is_distributeur()` gardent leur nom.

4.2 Git et GitHub

Élément	Convention	Exemple
Organisation / dépôt	kebab-case	`gresi-si` / `gresi`
Branches de travail	`type/numero-ticket-description`	`feat/42-scan-anti-rebond`, `fix/57-compteur-negatif`
Types de branche / commit	`feat`, `fix`, `refactor`, `test`, `docs`, `chore`, `db`	`db/31-table-affectations`
Messages de commit	Conventional Commits, en français, à l'impératif	`feat(scan): ajoute l'anti-rebond de 3 s`
Titre de PR	Identique au commit principal	`feat(inscription): vérifie le matricule`
Tags de version	SemVer	`v0.1.0` (prototype S4), `v1.0.0` (production S8)
Labels	`brique:web`, `brique:scan`, `brique:bdd`, `prio:must`, `prio:should`, `bloquant`	—

4.3 Base de données

Élément	Convention	Exemple
Tables	snake_case, **pluriel**, français sans accents	`activites`, `inscriptions`, `journal_audit`
Colonnes	snake_case, singulier	`intitule`, `qr_token`, `lieu`
Clé primaire	`id` (uuid, sauf journaux en `bigint identity`)	`id uuid primary key default gen_random_uuid()`
Clés étrangères	`<entite_au_singulier>_id`	`participant_id`, `activite_id`, `distributeur_id`
Horodatages	participe passé + `_le`, type `timestamptz`	`cree_le`, `modifie_le`, `valide_le`, `scanne_le`
Booléens	préfixe `est_`	`est_actif`, `est_publiee`
Types énumérés	snake_case singulier ; valeurs en minuscules	`creneau` : `matin`, `apres_midi`
Fonctions RPC	verbe à l'infinitif + complément	`choisir_activite`, `valider_scan`, `regenerer_qr`

Élément	Convention	Exemple
Paramètres / variables PL/pgSQL	préfixes `p_` / `v_`	`p_activite_id`, `v_resultat`
Triggers	`trg_<table>_<objet>` ; fonction `fn_<objet>`	`trg_inscriptions_verrou`, `fn_audit`
Contraintes	`uq_`, `ck_`, `fk_` + table + colonnes	`uq_inscriptions_participant_creneau`
Index	`idx_<table>_<colonnes>`	`idx_scans_activite_scanne_le`
Politiques RLS	phrase courte « rôle peut action objet »	`"participant lit ses inscriptions"`
Vues	préfixe `v_`	`v_participants_admin`
Actions du journal	`entite.verbe_au_participe`	`activite.annulee`, `qr.regenere`
Migrations	`AAAAMMJJHHMMSS_nom.sql` (généré par la CLI)	`20261010090100_tables.sql`
Edge Functions	kebab-case, verbe + objet	`verifier-etudiant`, `creer-personnel`

4.4 TypeScript et React

Élément	Convention	Exemple
Fichiers de composant	PascalCase.tsx, un composant exporté par fichier	`EcranResultat.tsx`
Pages	PascalCase + suffixe `Page`	`ScanPage.tsx`, `ConnexionPage.tsx`
Hooks	`use` + PascalCase, fichier `.ts`	`useScanner.ts`, `useActivites.ts`
Autres fichiers	camelCase.ts	`api.ts`, `queryClient.ts`, `format.ts`
Dossiers	kebab-case	`espace-participant`, `gestion-activites`
Types et interfaces	PascalCase, sans préfixe `I`	`Activite`, `ResultatScan`
Variables et fonctions	camelCase	`activiteEnCours`, `masquerMatricule()`
Booléens	préfixes `est` / `a`	`estConnecte`, `aDejaChoisi`
Constantes	SCREAMING_SNAKE_CASE	`DELAI_ANTI_REBOND_MS`, `PREFIXE_QR`
Props d'événement / gestionnaires	`on…` / `handle…`	`onScan`, `handleScan`
Clés React Query	tableau commençant par le domaine	`['activites', jour, creneau]`
Hooks de mutation	`use` + verbe	`useChoisirActivite()`
Exports	exports nommés (pas de `export default` sauf pages)	`export function Bouton()`

4.5 Styles

Les couleurs de la charte sont déclarées une seule fois dans `styles/index.css` (Tailwind v4, bloc `@theme`) et utilisées par leur nom :

Jeton	Valeur	Usage
`gresi-bleu`	#085CA6	Couleur principale, en-têtes, boutons
`gresi-bleu-2`	#0869A6	Survols, liens
`gresi-vert`	#7DBF4E	Succès, scan valide
`gresi-orange`	#F2780C	Alertes, scan refusé, actions secondaires
`gresi-gris`	#F2F2F2	Fonds
`gresi-gris-bleu`	#A7B6D0	Bordures, éléments désactivés

5. Variables d'environnement et secrets

Variable	Où	Secrète ?	Rôle
`VITE_SUPABASE_URL`	Vercel + `.env.local`	Non	URL du projet Supabase de l'environnement
`VITE_SUPABASE_PUBLISHABLE_KEY`	Vercel + `.env.local`	Non	Clé publique (« publishable », ou « anon » selon le projet). Protégée par la RLS.
`VITE_ENV`	Vercel + `.env.local`	Non	`dev`, `preprod` ou `prod` (bandeau d'environnement)
`VITE_DELAI_ANTI_REBOND_MS`	Vercel + `.env.local`	Non	Défaut 3000
`VITE_DUREE_RESULTAT_MS`	Vercel + `.env.local`	Non	Défaut 1500
`SUPABASE_URL`, `SUPABASE_SERVICE_ROLE_KEY`	Edge Functions (automatiques)	**Oui**	Accès administrateur, **uniquement** côté serveur
`VERIF_ETUDIANT_MODE`	Secrets Supabase	Non	`referentiel` ou `api`
`INPHB_API_URL`, `INPHB_API_CLE`	Secrets Supabase	**Oui**	Accès à l'API scolarité (si disponible)
`SMTP_HOTE`, `SMTP_PORT`, `SMTP_UTILISATEUR`, `SMTP_MOT_DE_PASSE`, `EMAIL_EXPEDITEUR`	Secrets Supabase + réglages Auth	**Oui**	Envoi des emails
`URL_APPLICATION`	Secrets Supabase	Non	Liens dans les emails
`SUPABASE_ACCESS_TOKEN`, `SUPABASE_PROJECT_ID_PROD`, `SUPABASE_DB_PASSWORD_PROD`	Secrets GitHub (environnement `production`)	**Oui**	Migrations prod depuis la CI

```
# apps/web/.env.example — copier en .env.local (jamais commité)
VITE_SUPABASE_URL=http://127.0.0.1:54321
VITE_SUPABASE_PUBLISHABLE_KEY=a-remplacer
VITE_ENV=dev                    # dev | preprod | prod
VITE_DELAI_ANTI_REBOND_MS=3000
VITE_DUREE_RESULTAT_MS=1500
```

Règles sur les secrets

Aucun secret dans le dépôt, dans un message WhatsApp ou dans un ticket. Ils vivent dans le **gestionnaire de mots de passe partagé** et dans les réglages de Vercel, Supabase et GitHub.

`.env.local` est dans le `.gitignore`. Seul `.env.example` (valeurs factices) est commité.

Une clé exposée par erreur est **révoquée et remplacée immédiatement**, puis l'incident est signalé à l'équipe.

6. Modèle de données v2

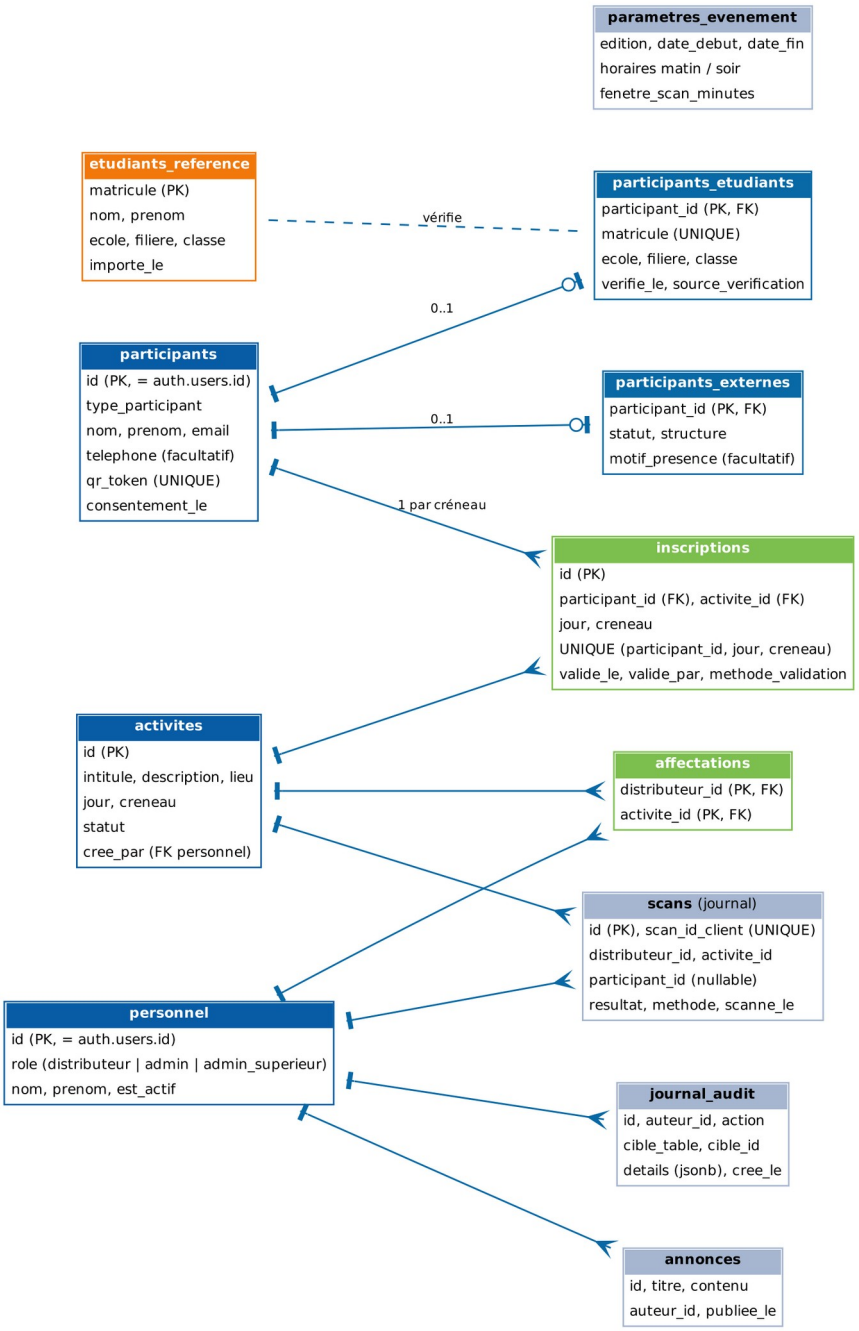


Figure 2 — Modèle de données v2

6.1 Types énumérés

Type	Valeurs
`type_participant`	`etudiant_inphb`, `externe`
`statut_externe`	`etudiant_hors_inphb`, `professionnel`, `visiteur`, `autre`
`role_personnel`	`distributeur`, `admin`, `admin_superieur`
`creneau`	`matin`, `apres_midi`

Type	Valeurs
statut_activite	brouillon, publiee, annulee
resultat_scan	valide, deja_utilise, non_inscrit, non_affecte, hors_creneau, qr_invalide
methode_validation	qr, manuelle

6.2 Tables

Toutes les tables ont `cree_le timestamptz not null default now()` ; les tables modifiables ont aussi `modifie_le` (mis à jour par trigger). Ces colonnes ne sont pas répétées ci-dessous.

participants

Colonne	Type	Contraintes / remarques
id	uuid	PK, = `auth.users.id`, `on delete cascade`
type_participant	type_participant	not null
nom, prenom	text	not null
email	text	not null, unique
telephone	text	facultatif
qr_token	uuid	not null, unique, `default gen_random_uuid()`
qr_regenere_le	timestamptz	renseigné par `regenerer_qr`
est_actif	boolean	not null, défaut `true`
consentement_le	timestamptz	not null

participants_etudiants / participants_externes

Table	Colonnes
participants_etudiants	participant_id (PK, FK) · matricule text not null **unique** · ecole, filiere, classe text not null · source_verification (api, referentiel, manuelle) · verifie_le timestamptz not null
participants_externes	participant_id (PK, FK) · statut statut_externe not null · structure text · motif_presence text

Renommage : l'ancienne table `participants_etrangers` devient `participants_externes`.

etudiants_reference

Colonne	Type	Remarques
matricule	text	PK
nom, prenom, ecole, filiere, classe	text	tels que fournis par l'INPHB
source	text	`import_csv` ou `api`
importe_le	timestamptz	—

Aucune politique RLS de lecture : seules la RPC `inscrire_participant` et l'Edge Function `verifier-etudiant` y accèdent. En mode `api`, la fonction met en cache dans cette table la fiche renvoyée par l'INPHB : la RPC n'a donc qu'un seul endroit où vérifier.

personnel, activites, affectations

Table	Colonnes
`personnel`	`id` (PK, = `auth.users.id`) · `role` role_personnel not null · `nom`, `prenom` · `est_actif` boolean
`activites`	`id` uuid PK · `intitule` text not null · `description` · `lieu` text not null · `jour` date not null · `creneau` creneau not null · `statut` statut_activite défaut `brouillon` · `cree_par` FK personnel
`affectations`	`distributeur_id` FK personnel · `activite_id` FK activites · PK (`distributeur_id`, `activite_id`) · `cree_par` FK personnel · contrainte : le personnel doit avoir le rôle `distributeur`

inscriptions

Colonne	Type	Contraintes / remarques
`id`	uuid	PK
`participant_id`	uuid	FK participants, not null
`activite_id`	uuid	FK activites, not null
`jour`, `creneau`	date, creneau	copiés depuis l'activité par trigger (dénormalisation pour la contrainte d'unicité)
—	—	**`uq_inscriptions_participant_creneau` UNIQUE (`participant_id`, `jour`, `creneau`)** → RG1
`valide_le`	timestamptz	null tant que non validé → RG5
`valide_par`	uuid	FK personnel
`methode_validation`	methode_validation	null tant que non validé

Changer d'activité (RG4) = **mettre à jour** `activite_id` de la ligne existante, via `choisir_activite`. L'index unique partiel de la V1 devient une contrainte unique simple. Le trigger existant de verrouillage après le début de la session est conservé.

scans, journal_audit, annonces, parametres_evenement

Table	Colonnes
`scans`	`id` bigint identity PK · `scan_id_client` uuid **unique** (RG13) · `distributeur_id` · `activite_id` · `participant_id` (null si QR invalide) · `resultat` resultat_scan · `methode` methode_validation · `scanne_le`
`journal_audit`	`id` bigint identity · `auteur_id` · `action` text · `cible_table` · `cible_id` · `details` jsonb · lecture seule (aucune politique d'écriture ; alimentée par `fn_audit`)
`annonces`	`id` uuid · `titre` · `contenu` · `est_publiee` · `publiee_le` · `auteur_id`
`parametres_evenement`	une seule ligne (`id smallint` PK, `check (id = 1)`) · `edition` · `date_debut`, `date_fin` · `matin_debut`, `matin_fin`, `apres_midi_debut`, `apres_midi_fin` (time) · `scan_ouverture_min` (défaut 30, avant la fin) · `scan_fermeture_min` (défaut 60, après la fin)

6.3 Fonctions RPC et Edge Functions

Fonction	Rôle	Appelée par
`inscrire_participant(...)`	Crée participant + fiche enfant + QR dans une transaction ; revérifie le matricule dans `etudiants_reference` (RG11)	Participant (après confirmation email)

Fonction	Rôle	Appelée par
`choisir_activite(p_activite_id)`	Crée ou remplace le choix du créneau (RG1, RG4)	Participant
`retirer_choix(p_jour, p_creneau)`	Supprime le choix avant le début de la session	Participant
`valider_scan(p_qr_token, p_activite_id, p_scan_id_client)`	Validation atomique et journalisation (§7)	Distributeur
`valider_manuellement(p_participant_id, p_activite_id, p_scan_id_client)`	Validation de secours tracée (RG14)	Distributeur, Admin sup.
`rechercher_inscrits(p_activite_id, p_recherche)`	Recherche par nom ou matricule parmi les inscrits	Distributeur
`stats_activites(p_jour)`	Inscrits, validés, taux de présence par activité	Distributeur (ses activités), Admin, Admin sup.
`liste_participants_admin(p_recherche, p_page)`	Liste avec matricule masqué	Admin
`regenerer_qr(p_participant_id)`	Nouveau jeton, ancien invalidé (RG15)	Admin sup.
Edge `verifier-etudiant`	Vérifie matricule + nom ; renvoie école, filière, classe	Public (limité en débit)
Edge `creer-personnel`	Crée un compte Auth + ligne `personnel`	Admin sup.
Edge `creer-participant-sur-place`	Création de compte à l'accueil	Admin, Admin sup.
Edge `envoyer-email`	Email aux inscrits d'une activité annulée	Trigger / Admin sup.

6.4 Matrice des droits (RLS)

L = lecture, E = écriture directe, RPC = écriture uniquement par fonction, — = aucun accès.

Table	Participant	Distributeur	Admin	Admin sup.
`participants` (+ enfants)	L soi	— (via RPC)	— (vue masquée)	L / E
`etudiants_reference`	—	—	—	L
`personnel`	—	L soi	L	L / E
`activites`	L publiées	L	L	L / E
`affectations`	—	L les siennes	L	L / E
`inscriptions`	L soi · RPC	L ses activités	via RPC stats	L
`scans`	—	L les siens · RPC	via RPC stats	L
`journal_audit`	—	—	—	L
`annonces`	L publiées	L publiées	L / E	L / E
`parametres_evenement`	L	L	L	L / E

6.5 Ce qui change par rapport aux scripts existants

Script V1	Évolution v2
`001_schema.sql`	Ajout de `affectations`, `journal_audit`, `parametres_evenement`, `etudiants_reference`, `annonces` ; `qr_token` uuid opaque ; renommage `participants_etrangers` → `participants_externes` ; contrainte unique simple sur les inscriptions ; colonnes de validation (`valide_le`, `valide_par`, `methode_validation`).
`002_rls_policies.sql`	Politiques pour les nouvelles tables ; `inscriptions` lisibles par les distributeurs affectés ; aucune lecture de `etudiants_reference`.
`003_functions.sql`	`valider_scan` : contrôle des affectations, fenêtre de scan, idempotence, retour du compteur. Nouvelles RPC (§6.3). Triggers d'audit.

6.6 Découpage en migrations

Aucune donnée réelle n'existant encore, les scripts V1 sont **réécrits en migrations v2 propres** (et non empilés en correctifs) :

Fichier	Contenu
`…090000_extensions_et_types.sql`	`pgcrypto`, types énumérés
`…090100_tables.sql`	Toutes les tables, clés, valeurs par défaut
`…090200_contraintes_et_index.sql`	Unicités, checks, index
`…090300_fonctions_utilitaires.sql`	`is_admin()`, `is_admin_superieur()`, `is_distributeur()`, `fn_modifie_le`
`…090400_rls.sql`	`enable row level security` + politiques
`…090500_rpc_metier.sql`	Fonctions du §6.3
`…090600_audit.sql`	`fn_audit` + triggers
`…090700_realtime.sql`	Ajout de `inscriptions` à la publication `supabase_realtime`

Règle des migrations

Une migration **appliquée sur `gresi-dev` n'est plus jamais modifiée**. Toute correction passe par une nouvelle migration. Avant la mise en production, on peut éventuellement les consolider — décision à prendre en S7.

7. Flux clés

7.1 Scan continu à plusieurs distributeurs

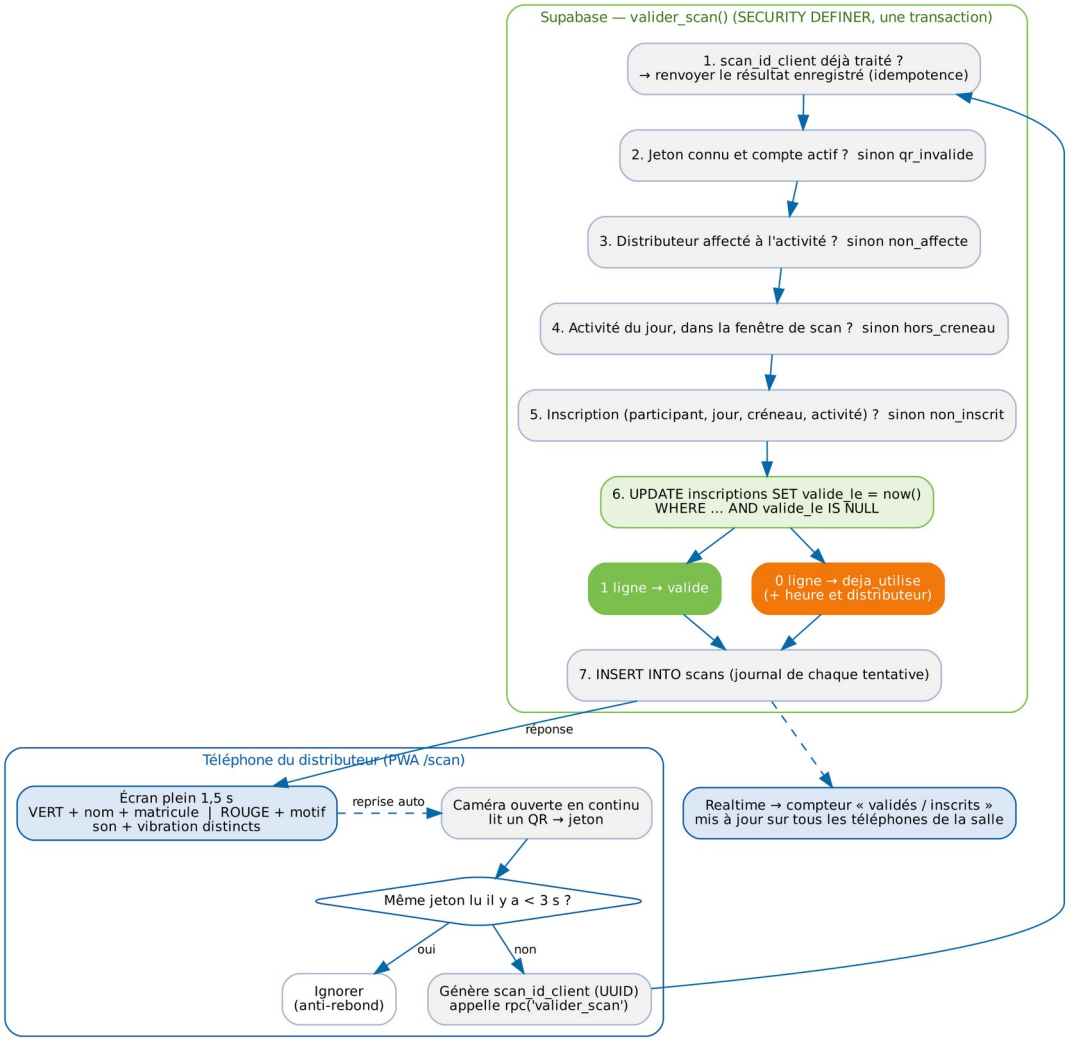


Figure 3 — Traitement d'un scan

```
-- Contrat de valider_scan (v2)
create or replace function public.valider_scan(
  p_qr_token        uuid,   -- extrait du QR « GRESI:1:<uuid> »
  p_activite_id     uuid,   -- activité sélectionnée par le distributeur
  p_scan_id_client  uuid    -- généré par le téléphone (idempotence)
) returns jsonb
language plpgsql security definer set search_path = public
as $$ ... $$;

-- Réponse (jsonb) :
-- { "resultat": "valide" | "deja_utilise" | "non_inscrit" | "non_affecte"
--               | "hors_creneau" | "qr_invalide",
--   "participant": { "nom": "...", "prenom": "...", "identifiant": "21IN***45" },
--   "valide_le": "2026-12-08T12:04:10Z", "valide_par": "Distributeur 2",
--   "compteur": { "valides": 187, "inscrits": 240 } }
```

Le cœur de la règle « une seule validation » (RG5, RG12) est une mise à jour conditionnelle : la première requête trouve la ligne, la seconde non.

```
update inscriptions
   set valide_le = now(), valide_par = auth.uid(), methode_validation = 'qr'
 where participant_id = v_participant_id
   and activite_id    = p_activite_id
   and valide_le is null          -- seule la 1re requête trouve encore la ligne
returning id into v_inscription_id;

if v_inscription_id is null then
  v_resultat := 'deja_utilise';
end if;
```

- **Format du QR** : `GRESI:1:<uuid>`. Le préfixe permet de rejeter immédiatement un QR étranger (affiche, autre application) sans appel réseau ; le `1` est la version du format.
- **Compteur** : `ScanPage` s'abonne aux mises à jour de `inscriptions` filtrées sur `activite_id`. Chaque validation, quel que soit le téléphone, incrémente le compteur de toute la salle.
- **Erreur réseau** : l'application renvoie le même `scan_id_client` ; le serveur reconnaît le scan et renvoie le résultat initial (RG13).

7.2 Création de compte étudiant

1. Le formulaire envoie matricule + nom à `verifier-etudiant`, qui consulte `etudiants_reference` (ou l'API INPHB puis met en cache).
2. Si la vérification réussit, école, filière et classe s'affichent en lecture seule.
3. `supabase.auth.signUp()` crée le compte Auth ; l'email de confirmation part via le SMTP configuré.
4. Au premier accès confirmé, l'application appelle `inscrire_participant`, qui **revérifie** le matricule côté base, contrôle son unicité et crée le participant avec son `qr_token`.

8. Plan de démarrage — semaine 0 (6 → 11 oct.)

Objectif : que le **lundi 12 octobre**, chaque développeur puisse cloner le dépôt, lancer la base et l'application en local en moins de 15 minutes, et prendre un ticket de S1. Aucun code fonctionnel n'est écrit avant que la *Definition of Ready* (§8.7) soit entièrement cochée.

Jour	Bloc	Responsable	Livrable
Mar. 6	Préparation	Stéphane	Cahiers des charges V2.0 diffusés à l'équipe
Mer. 7	Réunion de lancement + demandes externes	Les 3	Compte rendu, règles adoptées, emails envoyés
Jeu. 8	Comptes et espaces de travail	Les 3	Tous les accès fonctionnent
Ven. 9	Dépôt GitHub et squelette	Stéphane + Dev web	Dépôt structuré, protections actives
Sam. 10	Base de données	Stéphane	Migrations v2 sur `gresi-dev`, types générés
Dim. 11	Déploiement, CI, revue finale	Dev web + Dev scan, puis les 3	CI verte, Preview en ligne, tickets S1 créés

Les jours sont indicatifs et peuvent être décalés selon les cours ; l'ordre, lui, compte : chaque bloc dépend du précédent.

8.1 Réunion de lancement (mer. 7, 2 h)

1. Lecture commentée du cahier des charges fonctionnel V2.0 ; questions et objections (30 min).
2. Adoption des règles d'équipe (§9) et des conventions (§4) : chacun les valide explicitement (30 min).
3. Répartition des référents et des dossiers (§10) (10 min).
4. Décisions : nom de l'organisation GitHub, fournisseur SMTP, gestionnaire de mots de passe, canal de discussion (15 min).
5. Préparation des tickets de S1 : découpage en tâches de 2 jours maximum (35 min).

Le même jour : envoi de la demande à la DSI / scolarité de l'INPHB (API ou export du référentiel, autorisation écrite) et des questions au comité (date, salles et issues, nombre de distributeurs et modèles de téléphones).

8.2 Comptes et espaces de travail (jeu. 8)

Espace	Action	Vérification
GitHub	Activer la double authentification pour les 3 ; créer l'organisation `gresi-si` ; inviter les membres	Chacun voit l'organisation
GitHub Projects	Créer le tableau « GRESI — Réalisation » : colonnes *À faire*, *En cours*, *En revue*, *Terminé* ; champs *Brique*, *Semaine*, *Priorité* ; jalons S1 à S8	Un ticket test traverse les colonnes
Supabase	Créer l'organisation et deux projets `gresi-dev` et `gresi-prod` (région la plus proche disponible, à documenter) ; mots de passe de base dans le coffre	Les 3 accèdent aux deux projets
Vercel	Créer l'équipe et l'importer depuis GitHub (fait le 11)	Accès des 3
Coffre partagé	Gestionnaire de mots de passe d'équipe (ex. Bitwarden ; vérifier les limites de l'offre gratuite)	Les 3 lisent une entrée test
SMTP	Créer le compte (ex. Brevo), vérifier l'adresse d'expédition, noter les identifiants SMTP dans le coffre	Email de test reçu

Espace	Action	Vérification
Communication	Canal d'équipe avec 3 fils : *général*, *dev*, *alertes* (notifications GitHub)	Notification GitHub reçue
Docker + outils	Chacun installe Node (version LTS), Docker Desktop, Git, VS Code (+ extensions ESLint, Prettier, Tailwind)	`node -v`, `docker run hello-world`

8.3 Dépôt GitHub et squelette (ven. 9)

```
# Racine du monorepo
mkdir gresi && cd gresi && git init -b main
corepack enable && corepack prepare pnpm@latest --activate
node -v > .nvmrc
pnpm init
printf "packages:\n  - 'apps/*'\n  - 'packages/*'\n" > pnpm-workspace.yaml

# Application web
pnpm create vite apps/web --template react-ts
pnpm --filter web add @supabase/supabase-js @tanstack/react-query react-router zod react-hook-form
qrcode.react
pnpm --filter web add -D tailwindcss @tailwindcss/vite vite-plugin-pwa

# Paquet partagé + outils
mkdir -p packages/shared/src/{types,schemas}
pnpm add -Dw supabase prettier eslint typescript
```

1. Créer l'arborescence complète du §3 (dossiers vides avec un `.gitkeep` si nécessaire).
2. Configurer Tailwind v4 (`@import "tailwindcss"` + bloc `@theme` avec les couleurs GRESI), le routeur avec les routes du §3.3 (pages vides), le client `lib/supabase.ts` et `lib/env.ts`.
3. Configurer ESLint, Prettier, `.editorconfig`, `tsconfig.base.json` ; ajouter les scripts racine ci-dessous.
4. Rédiger `README.md` (démarrage en 10 minutes) et `CONTRIBUTING.md` (reprise du §9).
5. Ajouter `.github/PULL_REQUEST_TEMPLATE.md`, les modèles de tickets et `CODEOWNERS` (annexe B).
6. Créer le dépôt `gresi-si/gresi` (privé) et pousser `main`, puis créer la branche `production`.
7. Protéger `main` et `production` (règles GitHub) : PR obligatoire, 1 approbation, CI verte, pas de push forcé, pas de suppression ; revue des CODEOWNERS obligatoire.
8. Créer les labels (§4.2) et la première ADR : `ADR-001-monorepo.md`.

```
{
  "name": "gresi",
  "private": true,
  "scripts": {
    "dev": "pnpm --filter web dev",
    "build": "pnpm -r build",
    "lint": "eslint .",
    "typecheck": "pnpm -r typecheck",
    "format": "prettier --write .",
    "db:start": "supabase start",
    "db:stop": "supabase stop",
    "db:reset": "supabase db reset",
    "db:test": "supabase test db",
    "db:types": "supabase gen types typescript --local >
packages/shared/src/types/database.types.ts"
  },
  "devDependencies": { "supabase": "latest", "prettier": "latest", "eslint": "latest" }
}
```

8.4 Base de données (sam. 10)

```
pnpm supabase login
pnpm supabase init                                # crée supabase/config.toml
pnpm supabase start                               # base locale (Docker requis)
pnpm supabase migration new extensions_et_types   # répéter pour chaque migration
pnpm supabase db reset                            # rejoue migrations + seed.sql
pnpm supabase test db                             # tests pgTAP
pnpm db:types                                     # régénère les types TS
pnpm supabase link --project-ref <ref-gresi-dev>
pnpm supabase db push                             # applique sur gresi-dev
pnpm supabase functions new verifier-etudiant
pnpm supabase secrets set VERIF_ETUDIANT_MODE=referentiel
```

1. Initialiser `supabase/` et démarrer la base locale.
2. Réécrire `001`–`003` en migrations v2 (§6.6) ; écrire `seed.sql` : 3 admins sup., 1 admin, 4 distributeurs, 6 activités sur 2 jours, 30 participants fictifs, 20 lignes de `etudiants_reference`.
3. `supabase db reset` sans erreur ; écrire un premier test pgTAP (RG1) qui passe.
4. Générer `database.types.ts` et le commiter.
5. Lier et pousser sur `gresi-dev`.
6. Réglages Auth de `gresi-dev` : confirmation email activée ; *Site URL* et *Redirect URLs* (localhost + URL Preview Vercel) ; SMTP personnalisé ; longueur minimale du mot de passe à 10 ; modèles d'email en français (`supabase/templates`).
7. Vérifier que `inscriptions` est bien dans la publication Realtime.
8. Créer les 3 comptes admin supérieur de `gresi-dev`.
9. Ne **rien** créer encore sur `gresi-prod` hormis le projet lui-même (il sera alimenté par la CI en S8).

8.5 Déploiement et CI (dim. 11)

1. Importer le dépôt dans Vercel ; *Root Directory* = `apps/web` ; *Production Branch* = `production`.
2. Variables d'environnement Vercel : *Preview* → `gresi-dev` ; *Production* → `gresi-prod` (à remplir en S8).
3. Ajouter `vercel.json` (réécriture de toutes les routes vers `index.html`).
4. Ajouter `.github/workflows/ci.yml` et `deploy-db-prod.yml` (annexe B) ; créer l'environnement GitHub `production` avec approbation obligatoire et ses secrets.
5. Ouvrir une PR test : la CI passe, Vercel publie un lien Preview, la revue CODEOWNERS est demandée.
6. Dev scan : publier une page `/scan` minimale qui ouvre la caméra et affiche le texte d'un QR — premier signal pour le test de faisabilité de S1.

8.6 Revue finale (dim. 11, 30 min, les 3)

Chacun clone le dépôt **sur une machine propre** et suit le README. Si une étape bloque, le README est corrigé avant de clore la semaine 0. Les tickets de S1 sont ensuite assignés.

8.7 Definition of Ready — à cocher avant de coder

✓	Élément
☐	Cahier des charges fonctionnel V2.0 lu et validé par les 3 développeurs
☐	Règles d'équipe et conventions adoptées (compte rendu dans `docs/comptes-rendus/`)
☐	Organisation et dépôt GitHub créés, 2FA active pour tous, branches protégées
☐	Tableau GitHub Projects prêt, jalons S1–S8 créés, tickets S1 rédigés et assignés

✓	Élément
☐	Projets Supabase `gresi-dev` et `gresi-prod` créés ; secrets dans le coffre
☐	Migrations v2 appliquées en local et sur `gresi-dev` ; `seed.sql` fonctionnel ; types générés
☐	Au moins un test pgTAP passe en CI
☐	Application déployée en Preview, connectée à `gresi-dev`
☐	SMTP configuré : un email de confirmation reçu en préproduction
☐	Chacun lance base + application en local en moins de 15 minutes via le README
☐	Demandes envoyées à la DSI / scolarité INPHB et au comité

9. Règles d'équipe

9.1 Flux Git

1. Partir d'un ticket ; créer la branche depuis `main` à jour (`feat/42-…`).
2. Commits petits et fréquents (Conventional Commits).
3. Ouvrir une PR **en brouillon** dès le début pour rendre le travail visible.
4. PR prête : description, « Ferme #42 », captures si interface ; moins de 400 lignes modifiées si possible.
5. Revue par un autre développeur (et le référent CODEOWNERS) sous **24 h** ; CI verte obligatoire.
6. Fusion en *squash* ; la branche est supprimée.
7. Mise en production : PR `main` → `production` + tag, puis migrations via le workflow manuel.

9.2 Règles spécifiques à la base

- Toute modification de schéma passe par une migration revue par Stéphane.
- Toute nouvelle table : RLS activée et politiques **dans la même migration**.
- Toute nouvelle RPC ou règle de gestion : un test pgTAP dans `supabase/tests`.
- Après un changement de schéma : `pnpm db:types` et commit du fichier généré dans la même PR.
- Jamais de `select *` exposant des colonnes sensibles au client : passer par une vue `v_` ou une RPC.

9.3 Definition of Done

- Le ticket répond à ses critères d'acceptation.
- Code relu et fusionné ; CI verte.
- Testé en Preview, et sur un vrai téléphone Android si l'écran concerne le participant ou le scan.
- Aucune donnée réelle, aucun secret dans le code.
- Documentation mise à jour si une convention, une route ou une variable a changé.

9.4 Rituels

Rituel	Quand	Format
Point quotidien	Chaque jour avant 9 h	Message écrit : *Hier / Aujourd'hui / Bloquant*
Synchronisation	Mardi et vendredi, 30 min	Appel : avancement, blocages, arbitrages
Démo	Vendredi	Chacun montre ce qui est fusionné, en Preview
Rétrospective	Fin de semaine, 15 min	Ce qui marche / ce qu'on change ; une action concrète

9.5 Communication et décisions

- Une question technique qui engage l'équipe → ticket ou fil *dev*, pas en message privé.
- Une décision prise → ADR ou compte rendu dans `docs/`. Ce qui n'est pas écrit n'est pas décidé.
- Bloqué plus de 2 heures → on le signale dans le fil *dev*.

10. Répartition des responsabilités

Brique / dossier	Référent	Responsabilités
`supabase/`, `packages/shared/`, `.github/`, `scripts/`	Stéphane	Schéma, RLS, RPC, Edge Functions, tests SQL, CI, revue de toute PR touchant la base, test de charge
`apps/web` : `auth`, `activites`, `espace-participant`, `admin`, `annonces`, `components/`	Dev web	Écrans participant et admin, design system, routes, intégration des RPC
`apps/web/src/features/scan`, configuration PWA	Dev scan	Test de faisabilité S1, scan continu, sons, compteur, recherche manuelle, essais terrain, guide distributeur
`docs/`	Tous	Chacun documente ce qu'il livre ; Stéphane tient les cahiers des charges à jour

Si le plan B Expo est retenu en fin de S1, le Dev scan crée `apps/scan-mobile/` et réutilise `packages/shared` (types, constantes, format du QR).

Annexe A. Commandes utiles

Commande	Effet
`pnpm dev`	Lance l'application web en local
`pnpm db:start` / `pnpm db:stop`	Démarre / arrête la base locale
`pnpm db:reset`	Rejoue toutes les migrations + `seed.sql`
`pnpm db:test`	Lance les tests pgTAP
`pnpm db:types`	Régénère `database.types.ts`
`pnpm supabase migration new <nom>`	Crée un fichier de migration horodaté
`pnpm supabase functions serve`	Lance les Edge Functions en local
`pnpm lint` · `pnpm typecheck` · `pnpm build`	Ce que fait la CI

Annexe B. Fichiers modèles

.github/workflows/ci.yml

```
name: CI
on:
  pull_request:
  push:
    branches: [main]
jobs:
  web:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: pnpm/action-setup@v4
      - uses: actions/setup-node@v4
        with: { node-version-file: .nvmrc, cache: pnpm }
      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm typecheck
      - run: pnpm build
  base-de-donnees:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: supabase/setup-cli@v1
        with: { version: latest }
      - run: supabase db start      # applique toutes les migrations
      - run: supabase test db       # exécute supabase/tests/*.test.sql
```

.github/workflows/deploy-db-prod.yml

```
name: Migrations production
on: workflow_dispatch              # déclenchement manuel uniquement
jobs:
  push:
    runs-on: ubuntu-latest
    environment: production         # approbation requise dans GitHub
    env:
      SUPABASE_ACCESS_TOKEN: ${{ secrets.SUPABASE_ACCESS_TOKEN }}
      SUPABASE_DB_PASSWORD: ${{ secrets.SUPABASE_DB_PASSWORD_PROD }}
    steps:
      - uses: actions/checkout@v4
        with: { ref: production }
      - uses: supabase/setup-cli@v1
      - run: supabase link --project-ref ${{ secrets.SUPABASE_PROJECT_ID_PROD }}
      - run: supabase db push
```

.github/PULL_REQUEST_TEMPLATE.md

```
## Ce que fait cette PR
<!-- 2-3 lignes -->

Ferme #<numéro du ticket>

## Comment tester
1.
2.

## Checklist
- [ ] Testé en local (et sur un téléphone Android si l'écran est concerné)
```

```
- [ ] Aucune clé, mot de passe ou donnée réelle dans le code
- [ ] Nouvelle table → RLS activée + politiques dans la même migration
- [ ] Nouvelle RPC / règle → test pgTAP ajouté
- [ ] Types régénérés (pnpm db:types) si le schéma a changé
- [ ] Captures d'écran jointes si l'interface change
```

.github/CODEOWNERS

```
# Chaque PR touchant ces chemins demande l'avis du référent
/supabase/                          @<github-stephane>
/packages/shared/                   @<github-stephane>
/.github/                           @<github-stephane>
/apps/web/src/features/scan/        @<github-dev-scan>
/apps/web/                          @<github-dev-web>
```

Annexe C. Points ouverts

Point	Décision attendue	Échéance
PWA ou Expo pour le scan	Résultat du test de faisabilité sur les téléphones des distributeurs	Fin S1
Bibliothèque de lecture de QR	Comparer `BarcodeDetector` natif, `@zxing/browser` et `html5-qrcode` sur les mêmes téléphones	S1
Région des projets Supabase	Choisir la plus proche disponible et la documenter (données personnelles)	Semaine 0
Mode de vérification des étudiants	`referentiel` par défaut ; `api` si la DSI fournit un accès	S2 – S6
Consolidation des migrations avant production	Garder l'historique ou repartir d'une migration unique	S7